



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

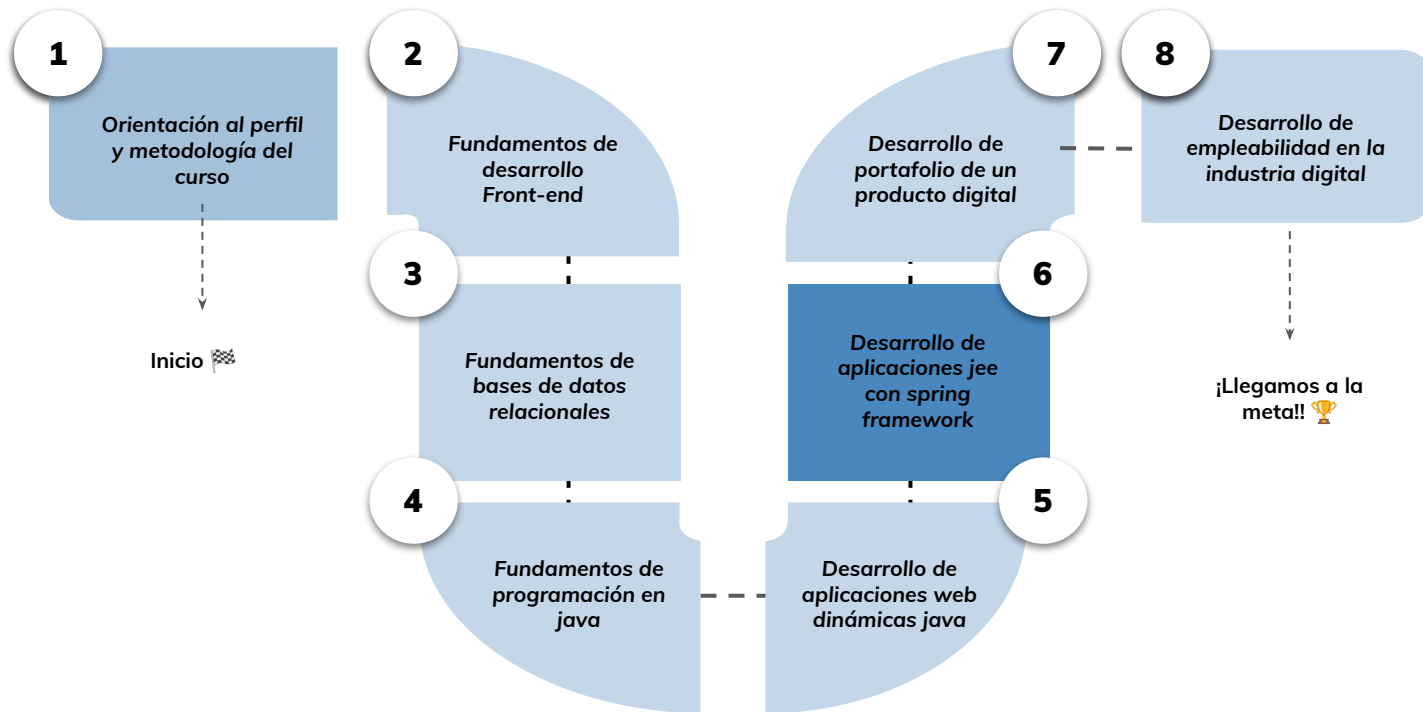


› El gestor de proyectos - Parte II

Aprendizaje Esperado 1: Utilizar un gestor de proyectos para la administración del ciclo de vida de un proyecto spring.

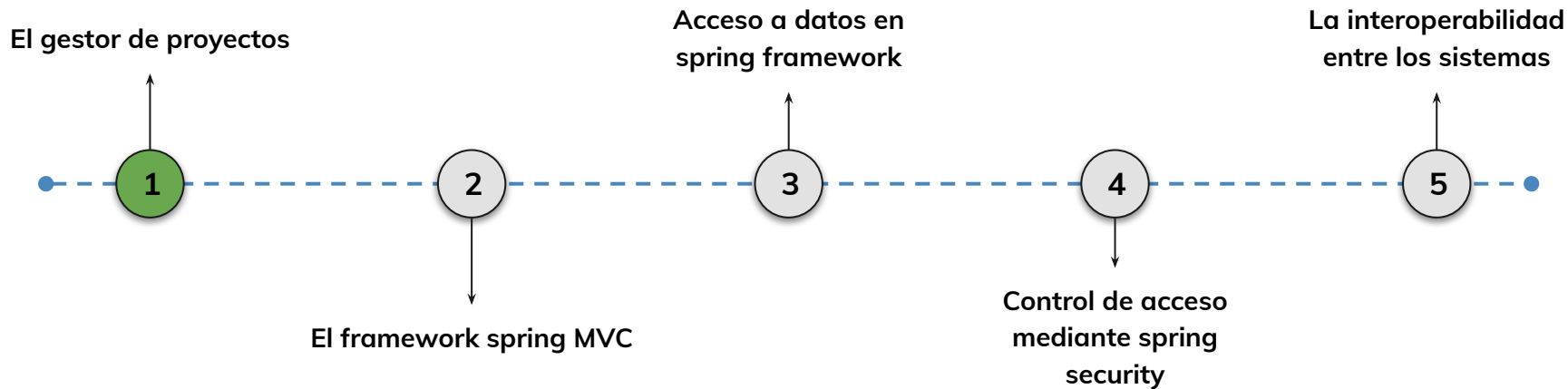
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



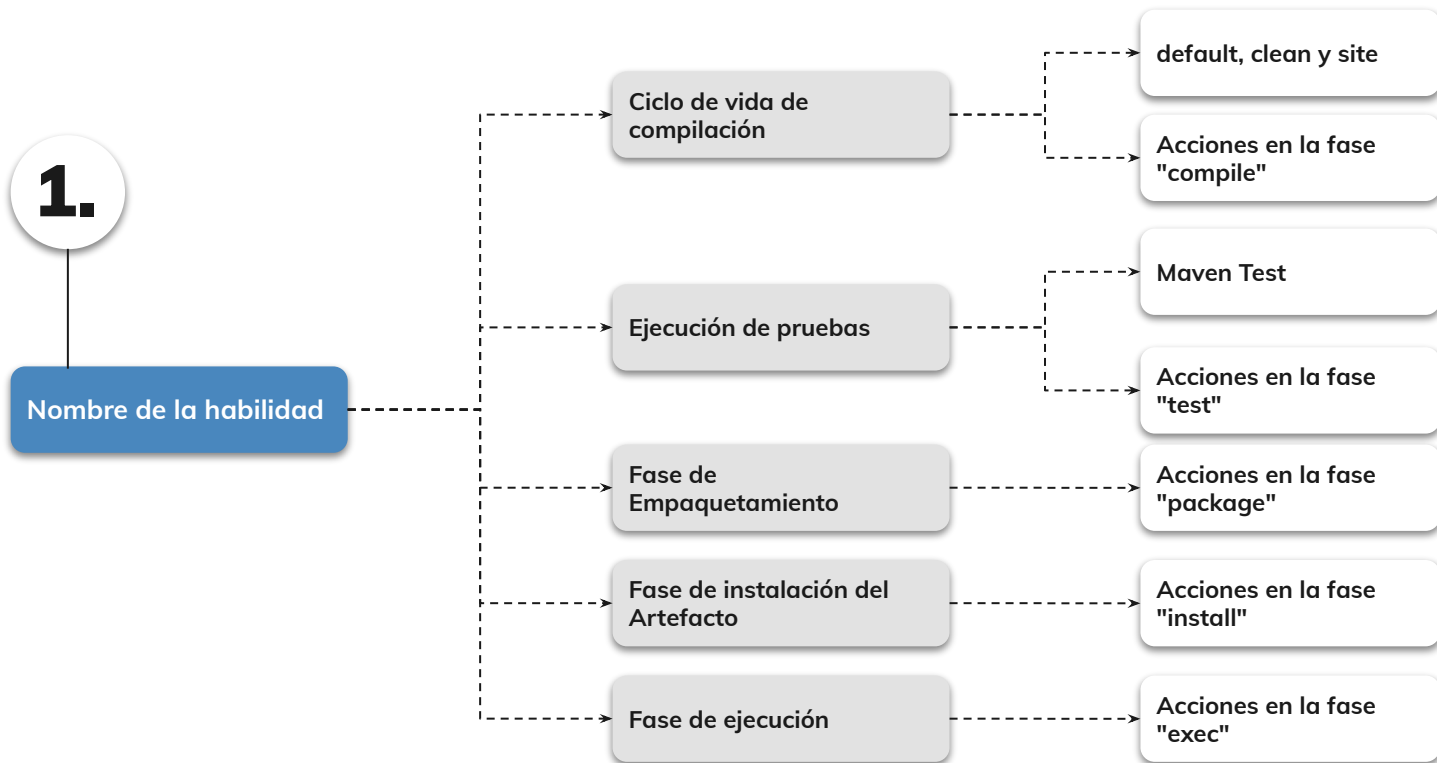
Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?



Objetivos de aprendizaje

¿Qué aprenderás?

- Aprender el concepto de ciclo de vida
- Conocer las etapas del ciclo de compilación
- Aprender el proceso de instalación y ejecución de Maven
- Conocer el proceso de empaquetamiento de Maven

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Comprender el concepto de gestor de proyectos
- Conocer Maven y el POM
- Aprender las particularidades de Spring Framework, Spring MVC y Spring Boot
- Comprender el concepto de dependencias
- Aprender las diferencias entre repositorio local y remoto

➤ Ciclo de vida de compilación

< > Ciclo de vida de compilación

Maven se fundamenta en el concepto central de ciclo de vida de construcción (**build lifecycle**). Esto significa que el proceso de construcción y distribución de un artifact (artefacto = **proyecto**) concreto está definido claramente.

Para la persona que quiera construir un proyecto significa que solamente es necesario aprender un pequeño conjunto de comandos para construir cualquier proyecto Maven, y el POM se asegurará de que obtenga los resultados deseados.



< > Ciclo de vida de compilación

Existen tres ciclos de vida en el sistema: **default** , **clean** y **site** .

- El ciclo de vida **default** controla el despliegue de tu proyecto.
- El ciclo de vida **clean** controla la limpieza de tu proyecto.
- El ciclo de vida **site** controla la creación del site de documentación de tu proyecto.



< > Ciclo de vida de compilación

Un ciclo de vida de construcción se compone de fases

Cada uno de estos ciclos de vida de construcción está definido por una lista diferente de fases (phases), donde cada fase representa un estado en el ciclo de vida.

Cada una de estas fases se ejecutan secuencialmente para completar el ciclo de vida.

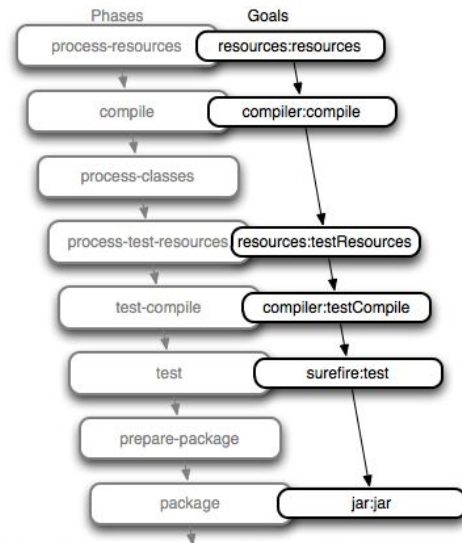
Clean Lifecycle	Default Lifecycle		Site Lifecycle
pre-clean	validate	test-compile	pre-site
clean	initialize	process-test-classes	site
post-clean	generate-sources	test	post-site
	process-sources	prepare-package	site-deploy
	generate-resources	package	
	process-resources	pre-integration-test	
	compile	integration-test	
	process-classes	post-integration-test	
	generate-test-sources	verify	
	process-test-sources	install	
	generate-test-resources	deploy	
	process-test-resources		



< > Ciclo de vida de compilación

Una fase de construcción se compone de objetivos de plugins

Sin embargo, aunque una fase de construcción es responsable de un paso específico en el ciclo de vida de construcción, el modo en que este se lleva a cabo puede variar. Esto se consigue declarando objetivos (goals) de plugins asociados a fases de construcción.



Note: There are more phases than shown above, this is a partial list



< > Ciclo de vida de compilación

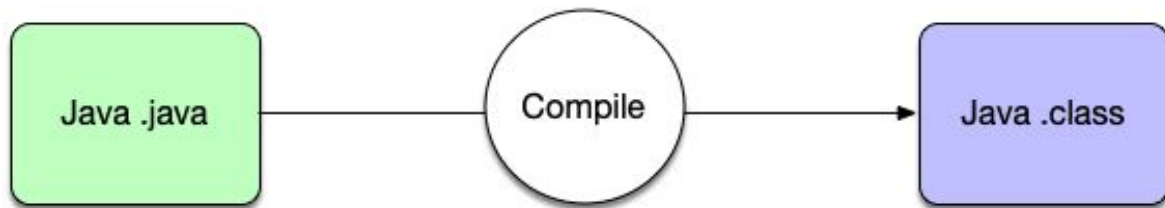
Un objetivo de un plugin representa una tarea específica que contribuye en la construcción y gestión de un proyecto. Puede estar asociada a cero o más fases de construcción. Un objetivo que no esté asociado a ninguna fase podría ejecutarse fuera del ciclo de vida de construcción mediante la invocación directa.

El orden de ejecución depende del orden en el que los objetivos y las fases de construcción sean invocados.



< > Ciclo de vida de compilación

El **ciclo de vida de compilación** de Maven es un proceso fundamental para la construcción y gestión de proyectos basados en Maven. Este ciclo de vida consta de tres fases principales: "**compile**", "**test-compile**" y "**process-resources**".



< > Ciclo de vida de compilación

Pasos y acciones en la fase "compile":

1- Recopilación de código fuente : En esta etapa, Maven busca los archivos fuente en el directorio de código fuente del proyecto, que suele ser "`src/main/java`". Este directorio contiene los archivos Java escritos por los desarrolladores. Maven recopila estos archivos y genera las clases Java compiladas correspondientes.

2- Gestión de dependencias : Antes de la compilación, Maven resuelve las dependencias del proyecto. Esto implica descargar las bibliotecas y dependencias necesarias desde los repositorios configurados. Maven garantiza que las versiones correctas de las dependencias estén disponibles para el proceso de compilación.



< > Ciclo de vida de compilación

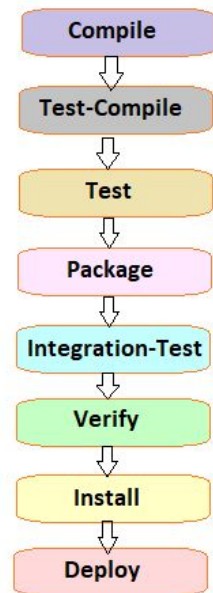
3- Compilación del código fuente : Una vez que las dependencias se han resuelto y los archivos fuente se han recopilado, Maven utiliza el compilador Java para compilar el código fuente en clases Java compiladas. Los archivos **.class** resultantes se almacenan en el directorio de salida, que suele ser "**target/classes**".

4- Generación de recursos compilados : Además de compilar archivos Java, Maven también puede procesar otros recursos, como archivos de configuración, archivos XML o archivos de propiedades. Los recursos se copian en el directorio de salida para que estén disponibles en la fase de empaquetado.



< > Ciclo de vida de compilación

5- Generación de archivos JAR/WAR/EAR : Aunque la generación de archivos JAR (Java Archive), WAR (Web Application Archive) o EAR (Enterprise Application Archive) no es parte de la fase "compile" en sí, la fase "compile" prepara el contenido que se incluirá en estos archivos. Las clases compiladas y los recursos procesados se utilizarán en la fase de empaquetado posterior.



< > Ciclo de vida de compilación

Beneficios y consideraciones de la fase "compile" en Maven:

- **Eficiencia y consistencia** : La fase "**compile**" de Maven automatiza la compilación de código fuente, lo que garantiza un proceso eficiente y consistente. Los desarrolladores no necesitan preocuparse por la configuración del compilador y pueden centrarse en escribir código.
- **Gestión de dependencias** : Maven se encarga de gestionar las dependencias, lo que garantiza que las bibliotecas y componentes necesarios estén disponibles y se utilicen en el proceso de compilación.



< > Ciclo de vida de compilación

Beneficios y consideraciones de la fase "compile" en Maven:

- **Compatibilidad con múltiples lenguajes y proyectos** : Aunque Maven es ampliamente utilizado en proyectos Java, también se puede configurar para admitir otros lenguajes y proyectos, lo que lo convierte en una herramienta versátil para la compilación.
- **Integración con herramientas de desarrollo** : Maven se integra con varias herramientas de desarrollo y entornos de desarrollo integrados (IDE), lo que facilita la gestión del ciclo de vida del proyecto y la compilación del código fuente.



< > Ciclo de vida de compilación

En resumen, la fase "compile" en el ciclo de vida de compilación de Maven es un proceso esencial que compila el código fuente de un proyecto, gestiona dependencias, procesa recursos y prepara el contenido que se utilizará en fases posteriores del ciclo de vida. Esta fase automatizada mejora la eficiencia y la consistencia en el desarrollo de proyectos, lo que hace que Maven sea una herramienta valiosa en el mundo del desarrollo de software.

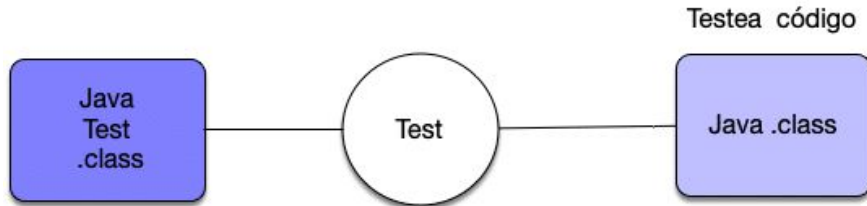


➤ Ejecución de pruebas

< > Ejecución de Pruebas

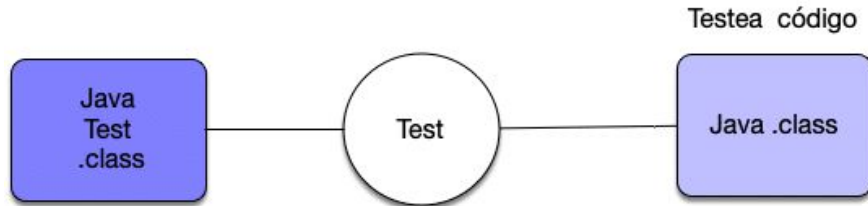
Maven Test:

Esta es otra de las fases principales en las cuales, **una vez se ha compilado el código, se ejecutan las pruebas unitarias** que se han construido para él . De esta forma nos aseguramos que nuestro código es correcto y no nos llevaremos ninguna sorpresa en producción.



< > Ejecución de Pruebas

El ciclo de vida de ejecución de pruebas en Maven es una parte fundamental del proceso de desarrollo de software. Esta fase, conocida como "**test**", se centra en la ejecución de pruebas unitarias y de integración en una aplicación.



< > Ejecución de Pruebas

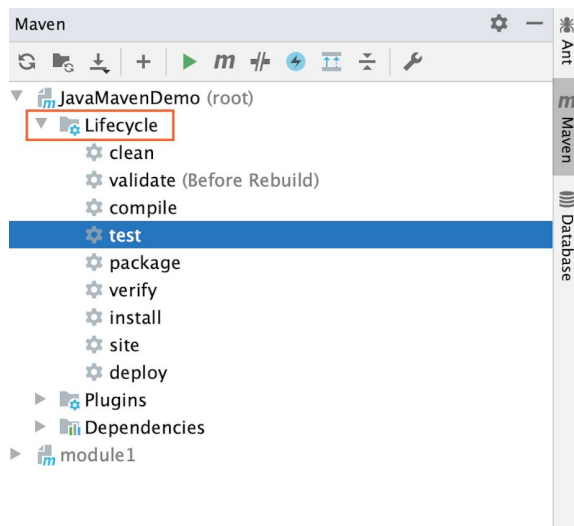
Pasos y acciones en la fase "test":

Su objetivo principal es llevar a cabo pruebas automatizadas en una aplicación para garantizar su calidad y funcionamiento correcto. Estas pruebas **pueden incluir pruebas unitarias, pruebas de integración y pruebas funcionales** .

1- Configuración de pruebas : Antes de ejecutar las pruebas, es necesario configurar el entorno de pruebas. Maven utiliza el directorio "**src/test**" para almacenar los archivos de código fuente de pruebas y los recursos relacionados con las pruebas. Las pruebas se organizan generalmente en paquetes y se crean archivos de prueba (por ejemplo, clases JUnit o TestNG) que contienen casos de prueba.



< > Ejecución de Pruebas



2- Compilación de pruebas: En esta etapa, Maven compila el código fuente de las pruebas. Los archivos de prueba, así como las clases y recursos de prueba, se compilan en clases Java compiladas (**.class**). Estas clases se almacenan en el directorio de salida de pruebas, que suele ser "**target/test-classes**".



< > Ejecución de Pruebas

3- Ejecución de pruebas: Una vez que las pruebas están compiladas, Maven inicia la ejecución de pruebas automatizadas. Esto implica la ejecución de casos de prueba individuales en busca de errores o problemas de lógica. Las pruebas pueden incluir pruebas unitarias, que se centran en componentes aislados, así como pruebas de integración que evalúan la interacción entre varios componentes.

4- Generación de informes de pruebas : Durante la ejecución de pruebas, Maven recopila información sobre las pruebas realizadas, los resultados obtenidos y cualquier error o fallo encontrado. Esta información se utiliza para generar informes detallados de pruebas. Los informes pueden estar en formatos como XML, HTML o texto y proporcionan una visión general de la calidad de la aplicación.



< > Ejecución de Pruebas

5- Cobertura de código : La fase "test" también puede incluir la medición de la cobertura de código. Esto implica determinar qué partes del código fuente se ejecutaron durante las pruebas y cuáles no. La cobertura de código es una métrica importante para evaluar cuán exhaustivas son las pruebas.

6- Detención de construcción en caso de fallos : Maven puede configurarse para detener la construcción en caso de que las pruebas fallen. Esto garantiza que no se produzca la generación de artefactos si la aplicación no pasa las pruebas.



< > Ejecución de Pruebas

Beneficios y consideraciones de la fase "test" en Maven:

- **Garantía de calidad:** La fase "test" de Maven asegura que las pruebas automatizadas se realicen de manera sistemática, lo que mejora la calidad del software y reduce la probabilidad de errores en la producción.
- **Retroalimentación rápida** : La ejecución de pruebas proporciona una retroalimentación rápida a los desarrolladores, lo que les permite corregir errores y problemas de manera oportuna.
- **Automatización y repetibilidad** : Las pruebas automatizadas son repetibles y se pueden ejecutar en múltiples entornos, lo que garantiza que el software se comporte de manera consistente.



< > Ejecución de Pruebas

Beneficios y consideraciones de la fase "test" en Maven:

- **Cobertura de código:** La medición de la cobertura de código ayuda a identificar áreas del código que no se prueban adecuadamente, lo que puede guiar la creación de pruebas adicionales.
- **Integración continua:** La fase "test" es esencial en un entorno de integración continua (CI) donde las pruebas se ejecutan automáticamente después de cada confirmación de código.
- **Mejora la colaboración:** Al garantizar que las pruebas sean parte del proceso de desarrollo, la fase "test" fomenta una mayor colaboración entre los miembros del equipo de desarrollo y mejora la confianza en la calidad del software.



< > Ejecución de Pruebas

En resumen, la fase "test" en el ciclo de vida de ejecución de pruebas de Maven es una parte crucial del proceso de desarrollo de software. Proporciona un enfoque sistemático y automatizado para garantizar la calidad y la confiabilidad del software, al tiempo que permite a los desarrolladores recibir retroalimentación temprana y continua sobre el estado de sus aplicaciones. La ejecución de pruebas automatizadas es una práctica esencial en el desarrollo de software moderno y contribuye significativamente a la entrega de software de alta calidad a los usuarios finales.



LIVE CODING

Test:

En este ejemplo en vivo, veremos la carpeta Test de un proyecto Maven para comprender de manera gráfica cómo se crean las carpetas y las pruebas de ejecución.

Tiempo: 15 minutos



Ejercicio N° 1

Compile y Test

Compile y Test

Manos a la obra: 🙌

En este ejercicio, vamos a crear un proyecto Java simple y escribir pruebas unitarias para él. Además, ejecutaremos los comandos Maven necesarios para realizar la compilación y la ejecución de pruebas.

Paso a paso: ✍️

1- **Configuración del entorno** : Asegúrate de tener instalado Java en tu sistema y Maven correctamente configurado.

Tiempo 🕒: 30 minutos

Compile y Test

2- **Creación de un proyecto Maven** : Abre una terminal y crea un directorio para tu proyecto. Luego, ejecuta el siguiente comando para crear un proyecto Maven:

```
mvn archetype:generate -DgroupId=com.ejercicio -DartifactId=maven-compile-test  
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

Esto creará una estructura de proyecto Maven básica con un directorio de código fuente ("src/main/java") y un directorio de pruebas ("src/test/java").

Compile y Test

3- **Escribir código fuente** : Dentro del directorio "src/main/java/com/ejercicio", crea una clase Java simple llamada "Calculadora" con un método para sumar dos números. Por ejemplo:

```
package com.ejercicio;

public class Calculadora {
    public int sumar(int a, int b) {
        return a + b;
    }
}
```

Compile y Test

4- Escribir pruebas unitarias: Dentro del directorio "src/test/java/com/ejercicio", crea una clase llamada "CalculadoraTest" para escribir pruebas unitarias. Utilizaremos el framework de pruebas JUnit para esto. Por ejemplo:

```
package com.ejercicio;

import org.junit.Test;
import static org.junit.Assert.*;

public class CalculadoraTest {

    @Test
    public void testSumar() {
        Calculadora calculadora = new Calculadora();
        int resultado = calculadora.sumar(2, 3);
        assertEquals(5, resultado);
    }
}
```

Compile y Test

5- Compilación y ejecución de pruebas: Abre una terminal en el directorio raíz de tu proyecto y ejecuta los siguientes comandos:

- Para compilar el código fuente, utiliza el siguiente comando:

`mvn compile`

- Maven compilará el código fuente y colocará las clases compiladas en "`target/classes`".
- Para ejecutar las pruebas unitarias, utiliza el siguiente comando:

`mvn test`

- Maven ejecutará las pruebas JUnit y generará un informe con los resultados.

Compile y Test

6- Verificar los resultados

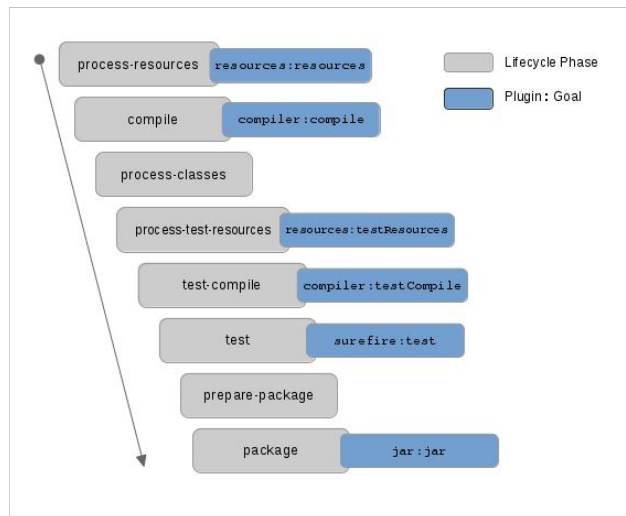
Maven generará un informe en el directorio "**target/surefire-reports**" con los resultados de las pruebas. Asegúrate de que las pruebas se ejecuten con éxito y que los resultados sean como se esperaba.

Este ejercicio te permite practicar la fase "compile" de Maven al compilar el código fuente de tu proyecto y la fase "test" al ejecutar pruebas unitarias. Puedes expandir el ejercicio agregando más pruebas o funcionalidad a tu código y asegurándose de que las pruebas continúen funcionando correctamente.

➤ Fase de Empaquetamiento

Fase de Empaquetamiento

El ciclo de vida de Maven consta de diversas fases diseñadas para guiar el proceso de construcción y gestión de proyectos. Una de las fases más esenciales es "**package**" (Empaquetamiento)



Fase de Empaquetamiento

Fase "package" (Empaquetamiento):

La fase "**package**" en el ciclo de vida de Maven se centra en empaquetar y ensamblar el proyecto en un formato específico, como un archivo **JAR**, **WAR** o cualquier otro tipo de artefacto, dependiendo del tipo de proyecto. **Empaquetar es una etapa esencial antes de desplegar o distribuir una aplicación**, ya que asegura que los componentes se agrupen de manera adecuada.



Fase de Empaquetamiento

Pasos y acciones en la fase "package":

1- Compilación y empaquetamiento: Maven comienza compilando el código fuente y recursos del proyecto. Luego, empaqueta estos archivos en un formato específico, dependiendo del tipo de proyecto (por ejemplo, JAR para aplicaciones Java, WAR para aplicaciones web, etc.).

2- Generación de informes y documentación: Esta fase también puede incluir la generación de documentación, como informes de construcción, documentación de API o cualquier otra documentación específica del proyecto.

Fase de Empaquetamiento

Pasos y acciones en la fase "package":

3- Generación de archivos de distribución: Si se requiere, Maven puede generar archivos de distribución para el proyecto, como archivos ZIP o tarball, que contienen la estructura de directorios y los archivos necesarios para la distribución de la aplicación.

4- Almacenamiento en el directorio de salida: Los artefactos generados y los archivos de distribución se almacenan en el directorio de salida, generalmente "target", que es un directorio temporal creado para la construcción del proyecto.

Fase de Empaquetamiento

Beneficios y consideraciones de la fase "package":

- **Agrupación de componentes:** La fase "package" garantiza que los componentes necesarios para la ejecución de la aplicación se empaqueten y se encuentren en una estructura adecuada.
- **Estandarización de la distribución:** Facilita la creación de archivos de distribución estandarizados para la aplicación, lo que simplifica el proceso de implementación y distribución.
- **Generación de documentación** : Puede incluir la generación de documentación para facilitar el uso y la comprensión del proyecto.
- **Fases posteriores:** Los artefactos generados en la fase "package" se utilizan en fases posteriores del ciclo de vida, como "install" para instalar en el repositorio local o "deploy" para desplegar en un repositorio remoto.

Fase de Empaquetamiento

Ejecutar la fase "package":

La fase "package" en Maven se ejecuta automáticamente como parte del ciclo de vida de construcción cuando utilizas comandos como **mvn install** o **mvn deploy** . No obstante, si deseas ejecutar únicamente la fase "package" por separado, puedes hacerlo mediante el siguiente comando:

```
mvn package
```

Fase de Empaquetamiento

Ejecutar la fase "package":

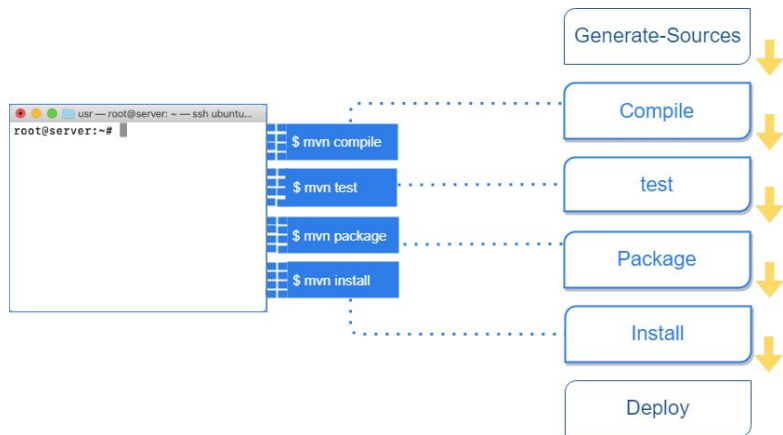
Maven compilará y empaquetará el proyecto de acuerdo con el tipo de proyecto (por ejemplo, creará un archivo JAR o WAR) y almacenará los artefactos en el directorio de salida, generalmente "target".

La ejecución de esta fase se integra de manera transparente con otras fases del ciclo de vida, lo que asegura la correcta preparación de los artefactos para su uso posterior.

➤ Fase de Instalación del Artefacto

Fase de Instalación del Artefacto

El ciclo de vida de Maven consta de una serie de fases que se ejecutan en un orden predefinido para gestionar el proceso de construcción y gestión de proyectos. Dos de las fases finales en este ciclo de vida son "**install**" (Instalación del Artefacto) y "**clean**" (Limpieza). A continuación, exploraremos en profundidad estas dos fases y su importancia en el proceso de desarrollo de proyectos Maven.





Fase de Instalación del Artefacto

Fase "install" (Instalación del Artefacto) :

La fase "**install**" en el ciclo de vida de Maven tiene como objetivo la instalación del artefacto generado en el repositorio local. Un "**artefacto**" en Maven se refiere a un elemento construido, como un archivo JAR, WAR o cualquier otro componente necesario para el proyecto. La instalación del artefacto en el repositorio local garantiza que esté disponible para otros proyectos en la misma máquina y se puede utilizar como dependencia.

Fase de Instalación del Artefacto

Pasos y acciones en la fase "install":

1- Compilación y empaquetado: Antes de instalar el artefacto, Maven compila el código fuente y empaqueta los archivos generados en un formato específico, como un archivo JAR o WAR. Esto incluye la compilación de clases, recursos, y otros archivos necesarios.

2- Instalación en el repositorio local: Una vez que el artefacto se ha generado y empaquetado correctamente, Maven lo instala en el repositorio local. El repositorio local es un directorio en la máquina del desarrollador donde Maven almacena sus dependencias y artefactos. Estos artefactos pueden ser compartidos con otros proyectos y equipos en la misma máquina.



Fase de Instalación del Artefacto

Pasos y acciones en la fase "install":

3- Resolución de dependencias locales: Antes de instalar un artefacto, Maven verifica si las dependencias necesarias ya están presentes en el repositorio local. Si no lo están, Maven intentará descargarlas automáticamente desde un repositorio remoto y las almacenará en el repositorio local para su uso futuro.

4- Generación de informes: La fase "install" también incluye la generación de informes que documentan la instalación del artefacto. Estos informes pueden ser útiles para rastrear y documentar la construcción y la distribución del artefacto.

Fase de Instalación del Artefacto

Beneficios y consideraciones de la fase "install":

- **Compartir artefactos** : La fase "install" permite a los desarrolladores compartir artefactos entre proyectos en la misma máquina, lo que facilita la reutilización de componentes.
- **Aislamiento de dependencias** : Cada proyecto tiene su propio repositorio local, lo que evita conflictos de dependencias entre proyectos.
- **Reducción de la necesidad de conexión a Internet** : Al almacenar dependencias localmente, la fase "install" reduce la necesidad de acceder a repositorios remotos cada vez que se construye un proyecto.
- **Documentación y seguimiento** : La generación de informes proporciona documentación sobre la instalación del artefacto, lo que puede ser útil para realizar un seguimiento de las versiones y la historia de construcción del proyecto.

Fase de Instalación del Artefacto

Ejecutar la fase "install":

1- Abre una terminal o línea de comandos en el directorio raíz de tu proyecto Maven.

2- Ejecuta el siguiente comando:

```
mvn install
```

Fase de Instalación del Artefacto

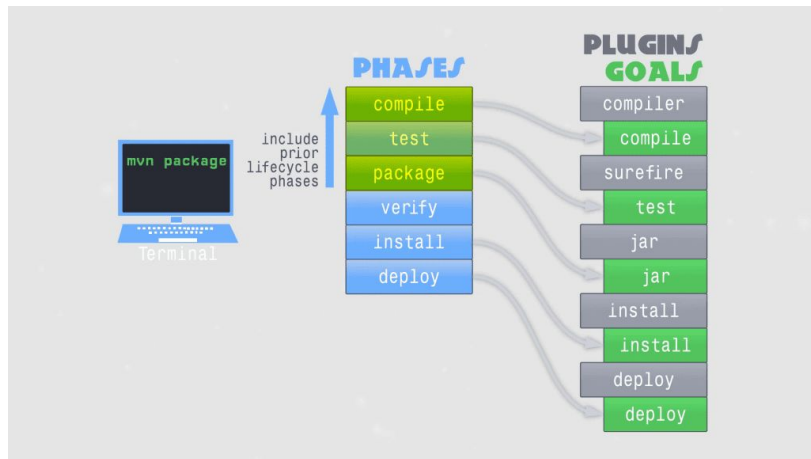
Ejecutar la fase "install":

3- Maven comenzará el proceso de construcción y ejecutará todas las fases necesarias, incluyendo "**compile**", "**test**", "**package**", y finalmente "**install**". En la fase "install", se instalarán los artefactos en el repositorio local.

4- Maven generará informes y mostrará resultados en la consola durante el proceso. Si todas las fases se completan con éxito, verás un mensaje de construcción exitosa al final.

➤ Fase de Ejecución

Fase de Ejecución



La fase "exec" en el ciclo de vida de Maven se utiliza para ejecutar programas o scripts que forman parte del proyecto. Esta fase es especialmente útil para ejecutar aplicaciones Java o scripts de línea de comandos como parte de la construcción del proyecto.

Fase de Ejecución

Pasos y acciones en la fase "exec":

1- Configuración de la ejecución : Antes de ejecutar la fase "exec", se configura el plugin "exec-maven-plugin" en el archivo pom.xml. Este plugin permite definir el programa o script que se ejecutará.

2- Ejecución de programas o scripts : Una vez configurado, Maven ejecuta el programa o script especificado. Puedes pasar parámetros y opciones personalizadas para la ejecución.

3- Resultados y salidas: Maven muestra los resultados y salidas de la ejecución en la consola. Puedes capturar la salida en archivos de registro si es necesario.

Fase de Ejecución

Pasos y acciones en la fase "exec":

1- Configuración de la ejecución : Antes de ejecutar la fase "exec", se configura el plugin "exec-maven-plugin" en el archivo **pom.xml** . Este plugin permite definir el programa o script que se ejecutará.

2- Ejecución de programas o scripts : Una vez configurado, Maven ejecuta el programa o script especificado. Puedes pasar parámetros y opciones personalizadas para la ejecución.

3- Resultados y salidas: Maven muestra los resultados y salidas de la ejecución en la consola. Puedes capturar la salida en archivos de registro si es necesario.

Fase de Ejecución

Beneficios y consideraciones de la fase "exec":

- **Automatización de tareas:** La fase "exec" permite automatizar tareas como la ejecución de aplicaciones, scripts o cualquier proceso que sea parte del proyecto.
- **Integración en el ciclo de vida:** La ejecución de programas o scripts se integra de manera natural en el ciclo de vida de construcción, lo que facilita la automatización de tareas relacionadas con el proyecto.
- **Control de la construcción:** Puedes controlar y personalizar la ejecución de programas o scripts como parte de la construcción del proyecto, lo que mejora la consistencia y la repetibilidad del proceso.
- **Captura de resultados:** Maven permite capturar los resultados y las salidas de la ejecución, lo que es útil para registrar y analizar los resultados de las tareas.

Fase de Ejecución

Ejecutar la fase "exec":

La fase "exec" se utiliza para ejecutar programas o scripts específicos como parte de la construcción del proyecto. Para ejecutar la fase "exec", sigue estos pasos:

1- Asegúrate de que el plugin "exec-maven-plugin" esté configurado en el archivo pom.xml de tu proyecto. Debes definir el programa o script que deseas ejecutar y configurar cualquier parámetro o opción necesaria. Aquí tienes un ejemplo de cómo se vería la configuración en el pom.xml:

Fase de Ejecución

```
<build>
  <plugins>
    <plugin>
      <groupId>org.codehaus.mojo</groupId>
      <artifactId>exec-maven-plugin</artifactId>
      <version>3.0.0</version>
      <executions>
        <execution>
          <id>my-execution</id>
          <goals>
            <goal>exec</goal>
          </goals>
          <configuration>
            <executable>java</executable> <!-- Nombre del ejecutable (por ejemplo, "java") -->
            <arguments>
              <argument>-jar</argument> <!-- Argumentos para el programa -->
              <argument>mi-aplicacion.jar</argument> <!-- Nombre del archivo ejecutable o script -->
            </arguments>
          </configuration>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>
```

Fase de Ejecución

Ejecutar la fase "exec":

2- Una vez que la configuración está en su lugar, puedes ejecutar la fase "**exec**" utilizando el siguiente comando:

```
mvn exec:exec
```

Maven ejecutará el programa o script definido en la configuración del plugin. Los resultados y las salidas de la ejecución se mostrarán en la consola.

Fase de Ejecución

Consideraciones adicionales:

- Puedes personalizar la configuración del plugin "exec-maven-plugin" en el archivo pom.xml para adaptarse a tus necesidades. Esto incluye la definición de ejecutables, argumentos y opciones específicas de la ejecución.
- La fase "exec" es especialmente útil para ejecutar aplicaciones Java o scripts de línea de comandos como parte del proceso de construcción. Puedes integrar esta fase en el ciclo de vida de Maven para automatizar tareas de construcción adicionales, como la generación de documentación o la ejecución de scripts de despliegue.
- Ten en cuenta que Maven permite la ejecución de fases específicas y se integra con otras fases del ciclo de vida de manera coherente. Esto facilita la automatización de tareas personalizadas en el proceso de construcción del proyecto.

LIVE CODING

Test:

En este ejemplo en vivo, realizaremos la compilación, el empaquetamiento, la instalación del artefacto y la ejecución del proyecto de ejemplo de la clase anterior.

Tiempo: 15 minutos



Ejercicio N° 1

mvn-install

mvn-install

Manos a la obra: 🙌

¡Finalmente vamos a unificar lo aprendido! Vamos a continuar trabajando sobre el proyecto creado en la clase anterior para una Wallet. En este caso, vamos a empaquetar e instalar el artefacto.

Paso a PAso: 📝

1- Compilar y empaquetar el proyecto: En la terminal, ve al directorio raíz del proyecto y ejecuta el siguiente comando para compilar y empaquetar el proyecto en un archivo JAR:

`mvn package`

Tiempo 🕒: 15 minutos

mvn-install

Paso a Paso: 📝

Maven compilará el código fuente y empaquetará el proyecto como un archivo JAR. El archivo JAR resultante se almacenará en el directorio "target" dentro del proyecto.

2- Instalar el proyecto en el repositorio local: Para instalar el proyecto en el repositorio local de Maven, utiliza el siguiente comando:

```
mvn install
```

Maven copiará el archivo JAR generado en la fase "package" en el repositorio local, que suele encontrarse en tu directorio de usuario, en la carpeta **".m2/repository"**. El artefacto estará disponible para otros proyectos en la misma máquina.



Momento:

Time-out!



5 -10 min.



¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Comprender el concepto de ciclo de vida de compilación y pruebas
- ✓ Aprender a utilizar los comandos “compile” y “test”
- ✓ Comprender el concepto de la fase de empaquetamiento
- ✓ Aprender a utilizar los comandos “install” y “exec”



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

